LAB S08. SAGA Y CIRCUIT BREAKER

Javier Velasco Pacheco

Grupo 2

Sistemas Distribuidos 2027-1

1.El problema: una transferencia incompleta

Se realizó una transferencia de $200 MXN entre las cuentas de Ana y Beto, configurando una pausa de 20 segundos entre el cargo y el abono. Durante la operación se detuvieron las tres réplicas del servicio cuenta, provocando un error HTTP 504 porque no había instancias disponibles para completar el abono. Finalmente, se reiniciaron las réplicas para comprobar su recuperación y observar un escenario de fallo .

Antes de realizar la transferencia, la cuenta de Ana (0021800000000000001) tenía un saldo de $1,000, mientras que la cuenta de Beto (0021800000000000002) tenía $500. Después de realizar la transferencia de $200, el saldo de Ana disminuyó a $800 y el de Beto manteniendose a $500 debido a la falla de abono.

```
PS D:\mexi-banco> curl.exe -i http://localhost:8080/api/cuentas/002180000000000001
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 06a29cc9c126
Content-Type: application/json
Date: Fri, 16 Oct 2026 20:55:26 GMT

{"id":9,"clabe":"002180000000000001","titular":"Ana","saldo":1000.00}
PS D:\mexi-banco> curl.exe -i http://localhost:8080/api/cuentas/002180000000000002
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: b914643e2df5
Content-Type: application/json
Date: Fri, 16 Oct 2026 20:55:42 GMT

{"id":10,"clabe":"002180000000000002","titular":"Beto","saldo":500.00}
PS D:\mexi-banco>
```

Figura. Saldos antes de la prueba.

```
PS D:\mexi-banco> curl.exe -i http://localhost:8080/api/cuentas/002180000000000001
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 801c2abdc017
Content-Type: application/json
Date: Fri, 16 Oct 2026 21:03:36 GMT

{"id":9,"clabe":"002180000000000001","titular":"Ana","saldo":800.00}
PS D:\mexi-banco> curl.exe -i http://localhost:8080/api/cuentas/002180000000000002
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 801c2abdc017
Content-Type: application/json
Date: Fri, 16 Oct 2026 21:03:43 GMT

{"id":10,"clabe":"002180000000000002","titular":"Beto","saldo":500.00}
PS D:\mexi-banco>
```

Figura. Saldos después de apagar y volver a encender cuenta.

Se realizó el cargo de $200 MXN a la cuenta de Ana, pero la transferencia falló al intentar abonar el dinero a la cuenta de Beto, por lo que el cliente recibió un error HTTP 504. El cargo ya se había guardado y no se revirtió automáticamente.

2. Mi Saga

La Saga orquestada permite coordinar una operación distribuida mediante varios pasos independientes. En Mexi-Banco, TransferenciaService funciona como orquestador porque decide qué operación ejecutar, actualiza el estado de la transferencia y determina qué hacer cuando ocurre un error.

La transferencia comienza en estado INICIADA, cuando se registra la solicitud. Después se realiza el cargo en la cuenta de origen y el estado cambia a CARGADA. A continuación, se intenta abonar el dinero en la cuenta destino. Si el abono se realiza correctamente, la transferencia pasa a ABONADA y finalmente a COMPLETADA.

Cuando el abono falla después de realizar el cargo, el orquestador inicia la compensación para devolver el dinero a la cuenta de origen. Si la devolución y su registro se completan correctamente, la transferencia termina en COMPENSADA. Si la compensación no puede completarse o no es posible confirmar su resultado, la transferencia queda en COMPENSACION_PENDIENTE, para que el problema pueda resolverse posteriormente.

Cada estado debe guardarse en la base de datos para conocer el último paso alcanzado, incluso si un servicio falla durante la operación.

2.1. Diagrama de estados de la transferencia

El siguiente diagrama representa el flujo general de la Saga y las alternativas que existen cuando el abono tiene éxito o falla.



Figura. Diagrama

Los estados permiten diferenciar una transferencia completada de una operación que requiere recuperar sus efectos. COMPENSADA indica que la devolución quedó confirmada, mientras que COMPENSACION_PENDIENTE indica que todavía existe una tarea por resolver.

2.2. Pasos de la Saga y sus compensaciones

Paso	Operación	Estado	Compensación
1	Registrar la solicitud de transferencia.	INICIADA	No aplica.
2	Descontar el monto de la cuenta de origen.	CARGADA	Devolver el monto descontado.
3	Abonar el dinero a la cuenta destino.	ABONADA	Si falla, iniciar la devolución.
4	Confirmar que la operación terminó correctamente.	COMPLETADA	No se requiere si todo salió bien.
5	Devolver el dinero después de un fallo.	COMPENSADA	Si la devolución o su registro fallan, dejar pendiente.
6	Recuperar una compensación incompleta.	COMPENSACION_PENDIENTE	Reintentar de forma segura hasta confirmar el resultado.

3. Código

3.1. Orquestación y actualización de estados

1. Enum de estados:

Java

```
public enum EstadoTransferencia {
    INICIADA, CARGADA, ABONADA, COMPLETADA,
    COMPENSANDO, COMPENSADA, COMPENSACION_PENDIENTE, RECHAZADA
}
```

Explicación: Este enumerador define cada paso y estado posible de la Saga. Permite que la base de datos sepa exactamente en qué paso se quedó la operación si un servicio se cae a la mitad.

2. Método para compensar:

Java

```
public void compensarTransferencia(Transferencia transferencia) {
    try {
        if (yaExisteDevolucion(transferencia.getId())) {
            transferencia.setEstado(EstadoTransferencia.COMPENSADA.name());
            transferenciaRepository.save(transferencia);
            return;
        }
        devolverMontoCuentaOrigen(transferencia);
        registrarMovimientoDevolucion(transferencia);
        transferencia.setEstado(EstadoTransferencia.COMPENSADA.name());
        transferencia.setDetalle("Saldo devuelto y movimiento registrado.");
    } catch (Exception e) {
        transferencia.setEstado(EstadoTransferencia.COMPENSACION_PENDIENTE.name());
    } finally {
        transferenciaRepository.save(transferencia);
    }
}
```

Explicación: Contiene la lógica para devolver el dinero si el abono falla. Es idempotente, ya que primero verifica si ya existe una devolución previa antes de ejecutar la llamada al servicio de cuenta y registrar el movimiento, evitando así devolver el dinero dos veces.

3. Método de rechazo definitivo:

Java

```
public void rechazoDefinitivo(Transferencia transferencia, String motivo) {
    transferencia.setEstado(EstadoTransferencia.RECHAZADA.name());
    transferencia.setDetalle(motivo);
    transferenciaRepository.save(transferencia);
}
```

Explicación: Se utiliza cuando la transferencia falla antes de realizar el cargo (por ejemplo, si la cuenta de origen no tiene fondos). Actualiza el estado a RECHAZADA y guarda el motivo, ya que al no haber cargo, no hay nada que compensar.

4. La compensación funcionando

En esta prueba se realizó una transferencia de $100 de la cuenta de Ana Demo a la cuenta de Beto Demo. El sistema rechazó el abono a la cuenta destino debido al fallo controlado que se configuró para la prueba. Por este motivo, la Saga intentó compensar la operación devolviendo el dinero a la cuenta de origen. Aunque el saldo de Ana se restauró correctamente, el servicio de movimientos respondió con un error HTTP 500 al intentar registrar la devolución, por lo que la transferencia quedó en estado COMPENSACION_PENDIENTE.

Al consultar los saldos, se comprobó que ambas cuentas terminaron con $1,000.00: Ana recuperó el dinero y Beto no recibió el abono. Sin embargo, el historial de movimientos solamente muestra los depósitos iniciales de ambas cuentas y la salida de $100 correspondiente a la transferencia. No aparece el movimiento que registra la devolución a Ana. Esto demuestra que, aunque los saldos quedaron correctos, el registro de la operación está incompleto. Por ello, el sistema conserva la transferencia como pendiente de revisión, ya que necesita verificar y completar el registro de la compensación sin devolver el dinero una segunda vez.

```
PS D:\mexi-banco-s08> docker compose exec db psql -U mexibanco -d transferencia -c "SELECT id, clav
 id |    clave_origen    |   clave_destino    | monto  |         estado          |
----+--------------------+--------------------+--------+-------------------------+------------------
-------------------------------------------------------
  7 | 999000000000000101 | 999000000000000102 | 100.00 | COMPENSACION_PENDIENTE | Abono rechazado:
ón. Revisar el historial: El servicio movimiento respondio 500
(1 row)
```

Figura. La consulta a la tabla transferencia, donde se observa el estado COMPENSACION_PENDIENTE y el detasdalle del error.

```
       clabe        |  titular  |  saldo
--------------------+-----------+---------
 999000000000000101 | Ana Demo  | 1000.00
 999000000000000102 | Beto Demo | 1000.00
(2 rows)
```

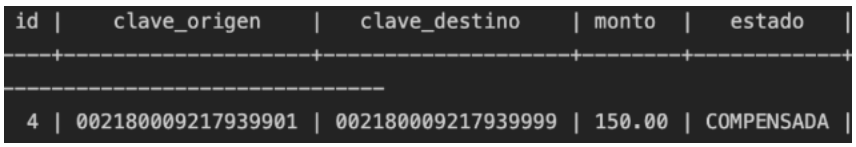
Figura La consulta a la tabla cuenta, donde ambas cuentas muestran un saldo de $1,000.00.

```
PS D:\mexi-banco-s08> docker compose exec db psql -U mexibanco -d movimiento -c "SELECT id, clabe_cuenta, tipo, monto, s
R BY id;"
 id |    clabe_cuenta    |        tipo          |  monto  | saldo_resultante |             referencia
----+--------------------+----------------------+---------+------------------+-------------------------------------
 26 | 999000000000000101 | DEPOSITO             | 1000.00 |          1000.00 | saldo inicial
 27 | 999000000000000102 | DEPOSITO             | 1000.00 |          1000.00 | saldo inicial
 28 | 999000000000000101 | TRANSFERENCIA_ENVIADA | -100.00 |           900.00 | Transferencia 7 a 999000000000000102
(3 rows)
```

Figura. Tabla movimiento, donde se observa la salida de $100 y la ausencia del registro de

Devolución

Al corregir el tipo de movimiento enviado al servicio, la Saga logró completar la compensación. La transferencia terminó exitosamente en estado COMPENSADA.



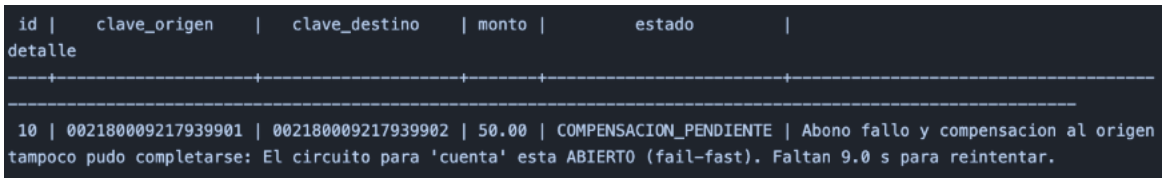
id	clave_origen	clave_destino	monto	estado
4	002180009217939901	002180009217939999	150.00	COMPENSADA

Figura. Compensada

5. Cuando la compensación también falla

Al consultar los saldos, Ana Demo y Beto Demo aparecieron con $1000.00 cada uno. Sin embargo, el historial de movimientos mostró los depósitos iniciales y el cargo de $100.00 realizado a Ana, sin el movimiento correspondiente a la devolución. Esto demuestra que restaurar el saldo y registrar el movimiento son operaciones que deben verificarse por separado.

Por este motivo, no sería correcto marcar la transferencia como COMPENSADA mientras el registro de la devolución no esté resuelto. El estado COMPENSACION_PENDIENTE permite identificar el problema y evita presentar la operación como completamente terminada.



id	clave_origen	clave_destino	monto	estado	detalle
10	002180009217939901	002180009217939902	50.00	COMPENSACION_PENDIENTE	Abono fallo y compensacion al origen tampoco pudo completarse: El circuito para 'cuenta' esta ABIERTO (fail-fast). Faltan 9.0 s para reintentar.

6. Circuit breaker

El siguiente código implementa manualmente el patrón Circuit Breaker en Java, sin utilizar bibliotecas externas como Resilience4j. Cuenta tres fallos consecutivos para pasar de CERRADO a ABIERTO, espera 10 segundos y después permite una llamada de prueba en estado SEMIABIERTO. Si la prueba falla, vuelve a abrir el circuito; si tiene éxito, regresa a CERRADO. También registra las transiciones para facilitar la comprobación de su funcionamiento.

```
package mx.mexibanco.clientes;

import java.util.concurrent.Callable;
import java.util.function.Predicate;

import org.springframework.stereotype.Component;

@Component
public class CircuitBreakerManual {

  public enum Estado {
    CERRADO,
    ABIERTO,
    SEMIABIERTO
```

```
}

private final int limiteFallos = 3;
private final long esperaMs = 10_000;

private Estado estado = Estado.CERRADO;
private int fallos = 0;
private long aperturaMs = 0;
private boolean pruebaEnCurso = false;

public <T> T ejecutar(
      Callable<T> llamada,
      Predicate<Throwable> esFalloDeDisponibilidad) throws Exception {

   boolean prueba = false;

   synchronized (this) {
      if (estado == Estado.ABIERTO) {
         if (System.currentTimeMillis() - aperturaMs < esperaMs) {
            throw new CircuitoAbiertoException(
                  "Circuit breaker ABIERTO: llamada a cuenta rechazada");
         }

         cambiarEstado(Estado.SEMIABIERTO);
      }

      if (estado == Estado.SEMIABIERTO) {
         if (pruebaEnCurso) {
            throw new CircuitoAbiertoException(
                  "Circuit breaker SEMIABIERTO: prueba en curso");
         }
         pruebaEnCurso = true;
         prueba = true;
      }
   }
   try {
      T resultado = llamada.call();
      registrarExito();
      return resultado;
   } catch (Exception e) {
      if (esFalloDeDisponibilidad.test(e)) {
         registrarFallo(prueba);
      } else {
         registrarExito();
      }
      throw e;
   }
}

private synchronized void registrarExito() {
   fallos = 0;
   pruebaEnCurso = false;

   if (estado == Estado.SEMIABIERTO) {
      cambiarEstado(Estado.CERRADO);
   }
}

private synchronized void registrarFallo(boolean prueba) {
   pruebaEnCurso = false;

   if (estado == Estado.SEMIABIERTO || prueba) {
      abrir();
      return;
   }

   if (estado == Estado.CERRADO) {
      fallos++;

      System.out.println(
            "[CIRCUIT BREAKER] Fallos consecutivos: " + fallos);

      if (fallos >= limiteFallos) {
         abrir();
      }
   }
```

```
    }

    private void abrir() {
      aperturaMs = System.currentTimeMillis();
      fallos = 0;
      cambiarEstado(Estado.ABIERTO);
    }

    private void cambiarEstado(Estado nuevo) {
      if (estado != nuevo) {
        System.out.println(
            "[CIRCUIT BREAKER] " + estado + " -> " + nuevo);
        estado = nuevo;
      }
    }

    public synchronized Estado getEstado() {
      return estado;
    }

    public static class CircuitoAbiertoException extends RuntimeException {
      public CircuitoAbiertoException(String mensaje) {
        super(mensaje);
      }
    }
  }
```

El Circuit Breaker sirve para evitar que transferencia siga intentando comunicarse con cuenta cuando el servicio está caído.

- CERRADO: las solicitudes se envían normalmente a cuenta.
- ABIERTO: después de tres fallos, bloquea temporalmente las llamadas para no seguir intentando conectarse a un servicio caído.
- SEMIABIERTO: después de 10 segundos, permite una llamada de prueba para comprobar si cuenta se recuperó.

Se enviaron tres solicitudes consecutivas de transferencia mientras el servicio cuenta estaba detenido. Las tres devolvieron el código HTTP 500 debido a que no había instancias disponibles. Los tiempos de respuesta fueron de 405 ms, 26 ms y 61 ms, respectivamente. Esto permitió comprobar que el sistema detectó los fallos consecutivos y activó el circuit breaker al alcanzar el límite de tres errores.

Los registros muestran que el servicio cuenta no tenía instancias disponibles y que el circuit breaker contabilizó los tres fallos consecutivos. Al alcanzar el límite establecido, el circuito cambió de CERRADO a ABIERTO, evitando continuar con llamadas a un servicio que no estaba disponible.

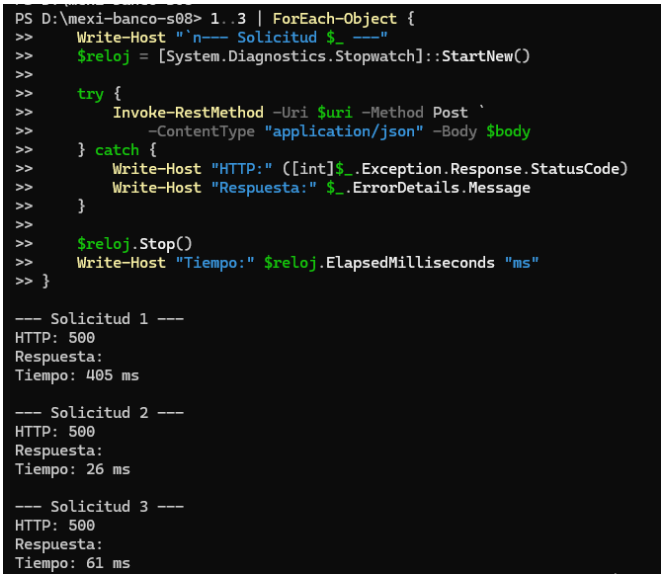


Figura. 3 Fallos

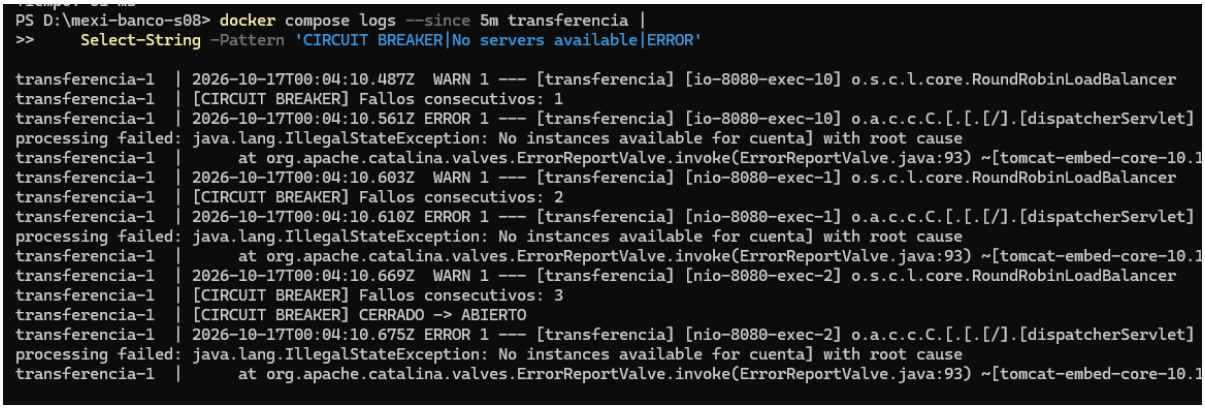


Figura. Cerrado a Abierto

Después, enviamos otra solicitud mientras el servicio seguía detenido. Como ya había transcurrido el tiempo de espera de 10 segundos, el circuito pasó a SEMIABIERTO e intentó comprobar si el servicio se había recuperado. La llamada volvió a fallar porque cuenta seguía sin instancias disponibles, por lo que el circuito regresó a ABIERTO.

```
PS D:\mexi-banco-s08> $reloj = [System.Diagnostics.Stopwatch]::StartNew()
PS D:\mexi-banco-s08>
PS D:\mexi-banco-s08> try {
>>     Invoke-RestMethod -Uri $uri -Method Post `
>>         -ContentType "application/json" -Body $body
>> } catch {
>>     Write-Host "HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     Write-Host "Respuesta:" $_.ErrorDetails.Message
>> }
HTTP: 500
Respuesta:
PS D:\mexi-banco-s08>
PS D:\mexi-banco-s08> $reloj.Stop()
PS D:\mexi-banco-s08> Write-Host "Tiempo:" $reloj.ElapsedMilliseconds "ms"
Tiempo: 86 ms
PS D:\mexi-banco-s08> docker compose logs --since 2m transferencia |
>>     Select-String -Pattern 'CIRCUIT BREAKER|No servers available'

transferencia-1  | [CIRCUIT BREAKER] ABIERTO -> SEMIABIERTO
transferencia-1  | 2026-10-17T00:06:41.464Z  WARN 1 --- [transferencia] [nio-8080-exec-3] o.s.c.l.core.RoundRobinLoadBalancer
transferencia-1  | [CIRCUIT BREAKER] SEMIABIERTO -> ABIERTO
```

Figura. Abierto a Semi-Abierto, y de Semi-Abierto a Abierto

Los registros muestran que, después de tres fallos consecutivos, el Circuit Breaker pasó de CERRADO a ABIERTO. Como el servicio cuenta seguía sin estar disponible, intentó comprobar su recuperación y volvió a abrirse. Finalmente, cuando las réplicas se reiniciaron, el circuito pasó de SEMIABIERTO a CERRADO, indicando que podía volver a realizar llamadas al servicio.

```
>>     Select-String -Pattern 'CIRCUIT BREAKER'

transferencia-1  | [CIRCUIT BREAKER] Fallos consecutivos: 1
transferencia-1  | [CIRCUIT BREAKER] Fallos consecutivos: 2
transferencia-1  | [CIRCUIT BREAKER] Fallos consecutivos: 3
transferencia-1  | [CIRCUIT BREAKER] CERRADO -> ABIERTO
transferencia-1  | [CIRCUIT BREAKER] ABIERTO -> SEMIABIERTO
transferencia-1  | [CIRCUIT BREAKER] SEMIABIERTO -> ABIERTO


PS D:\mexi-banco-s08> $uri = "http://localhost:8080/api/transferencias"
PS D:\mexi-banco-s08>
PS D:\mexi-banco-s08> $body = @{
>>     claveOrigen  = "00000000000000000"
>>     claveDestino = "00000000000000001"
>>     monto        = 1
>> } | ConvertTo-Json
PS D:\mexi-banco-s08>
PS D:\mexi-banco-s08> try {
>>     Invoke-RestMethod -Uri $uri -Method Post `
>>         -ContentType "application/json" -Body $body
>> } catch {
>>     Write-Host "HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     Write-Host "Respuesta:" $_.ErrorDetails.Message
>> }
HTTP: 404
Respuesta:
PS D:\mexi-banco-s08> docker compose logs --since 5m transferencia |
>>     Select-String -Pattern 'CIRCUIT BREAKER|No servers available'

transferencia-1  | [CIRCUIT BREAKER] ABIERTO -> SEMIABIERTO
transferencia-1  | 2026-10-17T00:06:41.464Z  WARN 1 --- [transferencia] [nio-8080-exec-3] o.s.c.l.core.RoundRobinLoadBalancer
transferencia-1  | [CIRCUIT BREAKER] SEMIABIERTO -> ABIERTO
transferencia-1  | [CIRCUIT BREAKER] ABIERTO -> SEMIABIERTO
transferencia-1  | [CIRCUIT BREAKER] SEMIABIERTO -> CERRADO
```

Figura. Circuit breaker

7. Lo que la saga no te da

La Saga orquestada permite coordinar una transacción entre varios servicios y garantiza que el sistema alcance un estado consistente eventualmente (mediante acciones compensatorias), pero

no garantiza que toda la operación sea atómica ni aislada como ocurre en una transacción tradicional de base de datos local.

Debido a que cada paso de la Saga hace un *commit* definitivo en la base de datos de su respectivo microservicio, los estados intermedios quedan expuestos y son visibles para el resto del sistema. En el escenario de nuestra prueba, si un usuario u otro servicio consulta la cuenta de Ana exactamente en el lapso de 20 segundos que existe entre el cargo inicial y la ejecución de la compensación, verá un saldo temporal de $800 en lugar de sus $1,000 reales.

Esta visibilidad de datos parciales puede provocar que el sistema o el usuario tomen decisiones incorrectas basadas en un estado temporal. Por ejemplo:

- Si Ana intenta pagar una compra urgente de $900 mediante otro servicio durante esos 20 segundos, la operación será rechazada por fondos insuficientes, a pesar de que la transferencia de $200 finalmente va a fallar y su dinero será restaurado.
- Si un proceso en lote (*batch*) calcula el saldo promedio mensual o genera un estado de cuenta en ese preciso instante, registrará un saldo menor al que realmente le corresponde al cliente al final del día.